

Подробный отчет о проделанной работе за июль 2026

Общий обзор задач, которыми занимался пользователь в рабочих проектах ОТТО и Kaufland

Общее резюме

В течение месяца основная работа была сосредоточена на развитии внутренних инструментов для управления товарами, интеграциями маркетплейсов и автоматизацией рутинных операций. Главный фокус был на том, чтобы сократить ручную работу при выставлении товаров, сделать процессы устойчивыми к сбоям, добавить прозрачность через логи и интерфейсы прогресса, а также подготовить систему к более массовой обработке товаров.

Работа охватывала backend, frontend, фоновые задачи, интеграции с внешними API, импорт данных, диагностику продакшен-ошибок и UX рабочих страниц. По сути, месяц был посвящен превращению разрозненных операций по товарам в более цельный операционный контур: загрузить данные, сопоставить товары, получить недостающую информацию, заполнить атрибуты, проверить ошибки, отправить изменения в маркетплейс и увидеть результат в интерфейсе.

Отдельно заметно, что задачи были не только про добавление новых функций, но и про доведение уже существующих процессов до рабочего состояния: разбор логов, поиск причин остановки batch-процессов, возможность продолжать выполнение с места сбоя, исправление неправильных запросов, уточнение схем данных и добавление тестов для новых внешних endpoints.

Ключевые результаты

- Развита рабочий контур выставления и обновления товаров для ОТТО, включая подготовку payload, работу с аккаунтами JV и XL, shipping profiles, availability и отправку данных обратно в маркетплейс.
- Добавлен и улучшен процесс AI-дозаполнения товарных атрибутов: выбор активных товаров, пакетная обработка, отправка изображений, заполнение только недостающих полей и отображение прогресса.
- Сделан импорт XLSX-данных и маппинг товаров между JV и XL по EAN, product reference и названию, включая отдельные таблицы для импортных строк и маппингов.
- Доработан внешний API-слой для получения товаров, категорий, атрибутов, shipping profiles, а также активации и деактивации по EAN.
- Существенно улучшены рабочие интерфейсы каталога, страницы товара, удаления товаров и управления ценами.
- Подключена observability-инфраструктура через Sentry и усилены логи для диагностики ошибок в production-like сценариях.
- В проекте Kaufland разбирались и исправлялись проблемы активации, деактивации, storefront-specific warehouse/shipping данных и некорректных API-ответов.

Автоматизация OTTO product workflow

Большой пласт работы был связан с тем, чтобы процесс создания и обновления товаров в OTTO стал управляемым, воспроизводимым и понятным. В фокусе были не отдельные ручные действия, а полный путь товара: от исходных данных и маппинга до подготовки финального payload и отправки в OTTO.

Разбиралась логика того, какие данные должен передавать внешний сервис для выставления товара, какие поля генерируются внутри системы, как формируются offer и unit identifiers, какие части payload обязательны, а какие могут быть вычислены локально. Это важно, потому что без четкого контракта между сервисами легко получить bad request на стороне маркетплейса или отправить неполный товарный объект.

В рамках этой работы уточнялась структура request body для endpoint выставления товара: какие поля нужны, какие типы данных ожидаются, какие значения приходят от внешнего сервиса, а какие могут быть созданы автоматически. Отдельно обсуждались EAN, SKU, product reference, offer id, unit id, pricing, availability, delivery/shipping настройки и товарные атрибуты.

Также развивалась логика batch-обработки: товары должны обрабатываться чанками, результаты должны сохраняться, а процесс не должен полностью ломаться из-за одной ошибки. Для долгих операций важными стали статусы, логирование, retry, polling и возможность продолжить выполнение с места остановки.

Что именно прорабатывалось

- Подготовка товаров к отправке в OTTO с учетом правил аккаунтов JV и XL.
- Маппинг товаров по EAN и fallback-стратегии, когда прямого соответствия нет.
- Сопоставление shipping profiles по названию между аккаунтами.
- Отправка availability после создания или обновления товара.
- Обработка ошибок OTTO API и перевод технических ошибок в понятный вид для интерфейса.
- Разделение prepare, submit preview и submit, чтобы перед отправкой можно было проверить итоговые данные.
- Работа с несколькими GPT API ключами для параллельной пакетной обработки.

Практический смысл этого блока в том, что процесс выставления товаров стал ближе к production-ready сценарию. Появилась возможность не просто нажать кнопку и ждать непонятный результат, а видеть состояние задачи, понимать где произошла ошибка, восстановиться после сбоя и продолжить работу без полного перезапуска всей партии.

AI-дозаполнение атрибутов товаров

Отдельным большим направлением стало заполнение недостающих атрибутов товаров через ИИ. Задача была сформулирована не как повторная генерация товара или категории, а именно как аккуратное дозаполнение missing attributes у уже существующих активных товаров. Это важное ограничение: система должна

сохранять уже заполненные данные и добавлять только то, чего не хватает.

Процесс должен был брать товары напрямую из ОТТО batch-запросами, выбирать только те, у которых одновременно присутствуют нужные активные статусы, и затем отправлять их в AI для анализа. Категория при этом не должна заново определяться, потому что товар уже существует в ОТТО и имеет свою категорию. ИИ используется как помощник для атрибутов, а не как полный генератор товара с нуля.

Важной частью стала передача изображений товара в AI. Для мебельных товаров многие атрибуты невозможно надежно заполнить только по тексту: цвета сиденья, корпуса, спинки, ножек или каркаса часто видны именно на картинке. Поэтому процесс был расширен так, чтобы использовать media assets и по возможности заполнять больше визуально определимых полей.

Детали работы с атрибутами

- ИИ получает только релевантные атрибуты конкретной категории, а не общий список из сотни лишних полей.
- Заполняются missing attributes, при этом существующие значения не должны затираться без необходимости.
- Для цвета добавлялись правила автозаполнения всех атрибутов со значением Farbe, когда это логично по основному материалу или изображению.
- Для вариаций отдельно учитывались Farbe и Material как основные dimensions, чтобы дополнительные цветовые поля не ломали логику вариантов.
- В интерфейсе показывается прогресс выполнения: сколько товаров обработано, какие товары завершены, где возникли ошибки и идет ли сейчас AI-этап.
- Добавлялись AI feed и status feed, чтобы было видно не только начало задачи, но и живое движение внутри процесса.

Результатом этого направления стал более понятный и полезный механизм массового улучшения качества карточек товаров. Он не заменяет оператора полностью, но снимает значительную часть ручной работы по заполнению однотипных и визуально определимых атрибутов.

Категории, атрибуты и переводы

В течение месяца отдельно разбиралась корректность базы категорий и атрибутов ОТТО. Проблема была в том, что для одной категории могло отображаться слишком много атрибутов, хотя фактически к конкретной категории должен относиться более ограниченный и точный набор. Из-за этого AI и интерфейс могли работать с лишними полями, что усложняло заполнение и снижало качество результата.

Поэтому прорабатывалась очистка и обновление таблиц, миграции, корректное связывание категорий с атрибутами и отображение этих данных в Swagger и frontend. Был сделан акцент на том, чтобы endpoints категорий и атрибутов возвращали именно те данные, которые нужны внешним потребителям и интерфейсу.

Также была задача отображать атрибуты на русском языке в интерфейсе. При этом немецкие оригинальные названия должны оставаться доступными, а русский перевод показываться рядом, например в скобках. Это улучшает работу оператора: можно быстрее понять смысл поля, не теряя точного ОТТО-названия.

Что было важно в этом блоке

- Сделать прямой endpoint категорий ОТТО видимым в Swagger.
- Очистить и обновить данные категорий и атрибутов в БД.
- Сделать так, чтобы для категории возвращались только связанные с ней атрибуты.
- Передавать в AI только релевантные атрибуты, чтобы не перегружать запрос и не ухудшать результат.
- Возвращать category id и attribute id там, где это нужно для надежной интеграции.
- Изменить получение атрибутов так, чтобы оно работало по category id, а не только по названию категории.
- Добавить русские переводы атрибутов и улучшить отображение длинных названий без кропа и визуального развала.

Этот блок напрямую влияет на качество всего товарного пайплайна. Если категории и атрибуты в базе неточные, то дальше страдают AI-заполнение, ручное редактирование, submit в ОТТО и внешний API. Поэтому работа была не декоративной, а фундаментальной для всей системы.

Инструкции сборки мебели

Еще один отдельный поток работы был посвящен атрибутам, связанным с инструкцией по сборке мебели. Нужно было понять, какие ОТТО-атрибуты отвечают за сборку, как они называются, какие значения допустимы, можно ли их заполнить через AI и какие поля нужны для разных категорий мебели.

В процессе был найден и проанализирован атрибут, связанный с языком инструкции: Sprachen Bedienungs-/Aufbauanleitung. Для массового исправления таких данных был нужен не интерфейсный сценарий, а terminal script, который можно запускать отдельно и контролировать через логи.

Скрипт должен был выбирать аккаунт, фетчить товары из ОТТО чанками, проверять каждый чанк на наличие нужного атрибута, заменять только значение на Deutsch (DE), отправлять измененные товары обратно в ОТТО, отслеживать статус задачи и корректно обрабатывать ошибки. Важными требованиями были retry, polling, информативные логи и возможность понять, действительно ли атрибут меняется правильно.

Смысловая ценность

- Появился отдельный инструмент для массового исправления конкретного ОТТО-атрибута без ручного редактирования товаров.
- Процесс стал контролируемым: можно видеть какие чанки обработаны, какие товары изменились, где была ошибка и что вернул ОТТО.
- Появилась база для будущей имплементации более широких правил по мебельным инструкциям для разных категорий.

Импорт XLSX и маппинг JV/XL

Крупный блок задач был связан с импортом XLSX-файлов и попыткой сопоставить товары между аккаунтами JV и XL. Главная бизнес-задача: по товару в JV найти соответствующий товар в XL, даже если прямого EAN-соответствия недостаточно. Первичная стратегия маппинга строилась по названию товара.

Для этого потребовалось не просто прочитать XLSX-файлы, а создать отдельные таблицы под импортные строки и маппинги. Это позволяет хранить исходные данные, повторно запускать обработку, анализировать причины несопоставленных строк и строить дополнительные стратегии маппинга.

Отдельной проблемой оказалось большое количество записей без source name. Было выяснено, что в исходном импорте не всегда хватает названия товара, поэтому появилась задача обогатить импортные строки: по product reference обращаться к OTTO endpoint, получать название товара и сохранять его в таблицу импорта. После этого можно строить более надежный name mapping.

Что входило в импортный контур

- Импорт всех XLSX-файлов из папки импорта в таблицу `otto_xlsx_import_rows`.
- Создание отдельных таблиц для EAN mapping и name mapping.
- Исключение строк Product Removed из маппинга, чтобы не связывать удаленные или невалидные товары.
- Обогащение импортных строк названием товара через OTTO API по product reference.
- Построение соответствий JV и XL по нормализованным названиям.
- Анализ причин, почему часть товаров не маппится автоматически.
- Подготовка данных к переносу из `otto_xlsx_import` в `products`.

Этот блок важен, потому что без надежного маппинга между JV и XL сложно делать массовое выставление, перенос цен, сопоставление availability и корректную работу с EAN. Даже если маппинг по названию не идеален, он создает основу для анализа и последующего улучшения правил.

Aftercool images и медиа товаров

Для улучшения каталога и AI-заполнения атрибутов понадобилась работа с изображениями товаров. Использовался Aftercool endpoint `/api/images-by-ean`, который принимает EAN как query-параметр и возвращает изображения товара. Нужно было решить, хранить ли эти данные заранее или получать их на лету.

Практическое решение двигалось в сторону кеширования: собрать изображения для товаров, сохранить их в отдельном месте и затем использовать при отдаче данных на frontend. Такой подход снижает количество повторных внешних запросов, ускоряет каталог и делает данные доступнее для AI-процессов.

Что делалось с изображениями

- Разбор формата ответа Aftercool images endpoint.
- Получение изображений по EAN через query-параметр.

- Подготовка сервиса для синхронизации изображений по импортным или товарным данным.
- Кеширование ссылок на изображения в отдельной таблице.
- Обновление продуктов media assets на основе найденных изображений.
- Использование изображений в каталоге и карточке товара.
- Передача изображений в AI для более точного заполнения визуальных атрибутов.

Каталог, карточка товара и рабочий UI

Значительная часть месяца была посвящена тому, чтобы интерфейсы стали удобнее для реальной операционной работы. Речь шла не о маркетинговом дизайне, а о плотных рабочих экранах, где оператор быстро видит список товаров, статусы, цены, изображения, ошибки и может массово редактировать данные.

Каталог товаров был переработан в сторону более компактной таблицы. Сначала интерфейс оказался слишком крупным, с большими отступами и неэффективным использованием места, поэтому потом он сжимался и адаптировался под рабочую плотность. Для карточки товара добавлялась галерея изображений, переключение между изображениями и fullscreen preview по клику.

Отдельный блок был связан с ценами. В каталоге нужен был редактор цен для одного товара и массовое редактирование для выбранных товаров. При изменении цены ОТТО должен обновляться `standardPrice`, а локальная база должна синхронизироваться с результатом. Также была добавлена логика пересчета UVP/PPЦ по формуле от цены.

UI-задачи, которые решались

- Редизайн страницы каталога и страницы товара под рабочий сценарий.
- Уменьшение таблицы и плотности строк, чтобы больше товаров помещалось на экране.
- Добавление изображений товара прямо в каталог.
- Добавление галереи и fullscreen просмотра изображений в карточке товара.
- Добавление выбора товаров в таблице для массовых действий.
- Добавление редактора цены в карточку товара.
- Добавление массового редактирования цены для выбранных товаров.
- Отображение сообщений об успешном обновлении и ошибках API.
- Улучшение отображения длинных названий и переводов, чтобы текст не обрезался и не ломал контейнер.

Эти изменения улучшают скорость ручной проверки и правки данных. Вместо того чтобы ходить по API или смотреть сырые JSON-ответы, оператор получает нормальный рабочий экран, где можно увидеть проблему, выбрать товары, изменить данные и отправить обновление.

Страница удаления товаров и деактивация

Отдельно прорабатывалась страница удаления товаров и логика деактивации по EAN. Нужно было сделать страницу понятнее, улучшить визуальное состояние результата, показывать успешные и неуспешные операции, а также разобраться с ошибками, когда marketplace API возвращает, что SKU неизвестен или товар не найден.

Для деактивации и активации добавлялись дополнительные логи, чтобы можно было понять, по какому EAN, аккаунту, storefront и endpoint произошла ошибка. Это

особенно важно в интеграциях маркетплейса, где один и тот же товар может иметь разные состояния в разных странах или аккаунтах.

External API layer

Внешний API-слой развивался как отдельный контракт для других сервисов. Нужно было дать корректные endpoints для получения товаров, shipping profiles, categories, attributes, а также для активации и деактивации товара по EAN. При этом endpoints должны быть видны и понятны в Swagger, принимать корректные параметры и возвращать данные в нужном формате.

В процессе исправлялась ошибка GET request body: браузерный fetch не допускает body у GET/HEAD запросов, поэтому параметры должны передаваться через query string. Также разбиралась ситуация, когда Swagger endpoint возвращал HTML 404 от Next.js, что указывает на неверную маршрутизацию или обращение не к тому backend endpoint.

Что было сделано в API-направлении

- Уточнение get products endpoint и перенос параметров GET-запроса в query params.
- Исправление shipping profiles external endpoint, чтобы он возвращал правильные данные.
- Добавление endpoint для получения categories без лишних category group данных.
- Добавление отдельного endpoint для получения attributes по category id.
- Возврат id категории и id атрибутов для надежной интеграции.
- Добавление activate и deactivate endpoints по EAN.
- Добавление integration-тестов для новых external endpoints.

Kaufland: активация, деактивация и storefront-логика

Помимо ОТТО, часть месяца была посвящена проекту Kaufland. Там основными темами были активация и деактивация товаров, ошибки Not Found, неверная работа с warehouse и units, а также необходимость учитывать разные storefront-данные для разных стран.

Проблема заключалась в том, что нельзя использовать один shipping id или warehouse для всех стран. Для каждого storefront могут быть свои требования, свои идентификаторы и свои условия. Поэтому требовалось посмотреть требования Kaufland API и исправить логику так, чтобы данные подставлялись корректно для каждой страны.

Также разбирались ошибки вида KeyError units при деактивации, 404 Not Found по storefront, сообщения об отсутствии warehouse и ситуации, когда в логах одно поле выглядит отсутствующим, хотя фактически проблема может быть в другом уровне payload.

Что решалось в Kaufland

- Добавление логов для активации и деактивации товара.
- Разбор причин 404 Not Found при запросах к storefront endpoints.
- Анализ ошибок по units и warehouse.
- Проверка корректности request body и query parameters.
- Исправление логики, где для каждой страны должны использоваться свои shipping и warehouse данные.
- Разбор ошибок quantity update, когда SKU неизвестен для API.

Эта работа была важна для надежности операций с товарами в нескольких странах. Чем больше storefront участвует в процессе, тем выше риск частично успешных операций: где-то товар обновился, где-то упал, где-то данные не совпали. Поэтому логирование и корректное разделение данных по странам стали ключевой частью задачи.

Инфраструктура, Sentry и диагностика

В течение месяца много внимания уделялось диагностике. Проверялись логи otto_bot.log, логи batch-процессов, причины остановки ОТТО XL batch, состояние instruction language update, ошибки mapper/classifier, проблемы dev окружения и frontend HMR.

Был подключен Sentry для backend и frontend. Это дает возможность видеть ошибки приложения не только через локальные логи, но и централизованно: исключения backend, ошибки frontend runtime, проблемы API, stack traces и контекст выполнения. После подключения отдельно разбиралось, что именно теперь может логироваться и как это помогает искать реальные production-проблемы.

Также разбирались инфраструктурные вопросы: почему creator page не открывается в Docker dev, почему возникает wss/HMR ошибка, безопасно ли убрать media из build context, почему Edge context занимает слишком много места, и как уменьшить лишний вес окружения.

Диагностические результаты

- Усилено логирование долгих фоновых задач.
- Добавлены детали ошибок для activate/deactivate сценариев.
- Появилась возможность видеть состояние batch-процессов и причины остановки.
- Подключен Sentry для backend и frontend.
- Разбирались причины HMR/WSS ошибок в dev окружении.
- Анализировался большой размер Edge context и безопасность удаления media из build context.

Тесты и проверка качества

Помимо разработки функциональности, в течение месяца добавлялись и исправлялись тесты. Отдельно проверялись OpenAPI endpoints, integration tests для external categories и shipping profiles, тесты instruction language service, тесты attribute generator и логики product variants.

Были случаи, когда тесты падали, и нужно было понять, является ли это реальной ошибкой в коде, проблемой ожиданий теста или следствием изменения контракта API. После исправлений проверялось, что endpoints возвращают правильные данные, а тесты соответствуют новой логике.

Итоговая оценка месяца

В целом месяц был посвящен не одной маленькой фиче, а системному развитию товарной платформы. Работа шла сразу на нескольких уровнях: данные, API, фоновые задачи, AI, интерфейс, логи, тесты и интеграции с внешними маркетплейсами.

Главный результат: процессы, которые раньше требовали ручной проверки, ручного маппинга, ручного исправления атрибутов и постоянного просмотра сырых логов, начали превращаться в управляемые инструменты. Появились таблицы импорта, маппинги, сервисы синхронизации, UI для прогресса, endpoints для внешних сервисов, редакторы в каталоге и диагностика через Sentry.

Работа также была направлена на масштабирование. Batch-обработка, retry, polling, несколько GPT-ключей, resume с места остановки и подробные статусы нужны именно для того, чтобы система могла обрабатывать не один товар вручную, а большие партии товаров с понятным контролем результата.

Если описывать одной фразой: в июле ты занимался построением полноценного рабочего контура для управления товарами на маркетплейсах, где данные импортируются, сопоставляются, обогащаются ИИ, проверяются, редактируются в интерфейсе и отправляются во внешние API с нормальным логированием и тестами.